

X-CDS: Explainable Clinical Decision Support

A to Z Comprehensive System Documentation & Technical Reference

Submitted by: Sahil Kumar (Reg: 23BAI10224)

VIT Bhopal University | School of Computer Science & Engineering (SCSE)

Advisor: Dr. Abdul Rahman

Document Objective

This guide outlines the complete codebase architecture, ingestion loops, re-ranking logic, and guardrail systems behind the Explainable Clinical Decision Support (X-CDS) framework. X-CDS mitigates the hallucination risks of Large Language Models in healthcare via hybrid retrieval and deterministic token-level validation loops.

1. Data Flow & System Architecture

The X-CDS system is organized as a pipeline that ensures clinical suggestions are directly anchored in validated literature. Below is the step-by-step breakdown of a query's lifecycle:

- **1. Clinical Query Submission:** The user (clinician) enters a question in the React split-screen dashboard.
- **2. Dual-Channel Retrieval:** The pipeline concurrently runs a dense semantic query (using BAAI/bge-small vector search in ChromaDB) and a sparse keyword query (using BM25 on the ingested corpus).
- **3. Reciprocal Rank Fusion (RRF):** Merges the results based on their reciprocal ranking order. This balances precise keyword identification (drug names) and vector semantics.
- **4. Cross-Encoder Re-ranking:** Top-N candidates are parsed by a Transformer Cross-Encoder model to determine a joint query-passage relevance score. The top-K documents are selected.
- **5. LangGraph Generation Node:** Generates a response with inline citations using Gemini (mapped via the new `langchain-google-genai` package and Vertex AI GCP credentials).
- **6. Token Overlap Citation Validator:** Sentinel logic splits the claims and ensures each citation matches the source document with at least 25% alphanumeric overlap (excluding stopwords).
- **7. Stateful Self-Correction Loop:** If validation fails, error details are mapped to state variables, routing the query back to the generation node for up to 3 correction attempts.
- **8. Independent Ragas Evaluation:** Measures pipeline performance using Faithfulness, Answer Relevancy, Context Precision, and Context Recall metrics.

2. Core Codebase File-by-File Breakdown

Below is a structural description of the core files inside the backend/app/ directory, explaining their exact code design and how they interface:

A. app/config/settings.py

Loads and validates environment configurations using Pydantic Settings. Crucially, it manages relative path resolution for the GCP credentials file (GOOGLE_APPLICATION_CREDENTIALS) relative to the project root, and sets up the process environment variables for Vertex AI API client discovery.

B. app/llm/generation.py

Implements RobustChatVertexAI using the new ChatGoogleGenerativeAI model class. It includes automated retry logic for transient Google Cloud API failures (429 rate limits, 503 unavailable) and falls back dynamically to gemini-3.1-flash-lite if the primary model is unavailable. It also wraps generation schema validation in a try-except block to return validation failure states to the Graph loop.

C. app/llm/graph.py

Constructs the stateful LangGraph graph. It compiles nodes for generation and validation, defining conditional edges (should_retry_generation) that route execution back to the generation node if citations fail alignment checks.

D. app/guardrail/validator.py

Implements the alphanumeric token overlap scoring. It parses bracketed inline citations (e.g. [1]), matches them against context document indices, splits paragraphs into sentences, filters stopwords, and flags claims that do not meet the CITATION_MIN_TOKEN_OVERLAP = 0.25 threshold.

E. app/guardrail/loop.py

Coordinates state transitions between the validator outputs and graph routing state, mapping validator logs to correction_feedback messages that prompt the LLM to rewrite and cite correctly on successive loop attempts.

F. app/eval/ragas_eval.py

Executes offline evaluation using the Ragas framework. It initializes Ragas wrappers using Vertex AI classes, computes metrics (Faithfulness, Relevancy, Context Precision, and Context Recall), aggregates results from NumPy formats, and outputs a JSON report (ragas_report.json).

3. Design Choices, Alternatives & Clinical Justifications

Deploying generative models in medicine requires prioritising factual safety and explainability over output speed or flexibility. The following analysis outlines our design choices and why they are appropriate for clinical settings:

Architecture / Node	Selected Choice	Discarded Alternative	Clinical Justification
State Management	LangGraph Cyclic Graph	LCEL Linear Chains	LCEL cannot loop. LangGraph allows cyclic self-correction loops when citations fail validation.

Information Retrieval	Hybrid Search + RRF	Pure Vector Search	Pure vector search misses critical exact terms (e.g. drug formulas). Keyword search ensures precision.
Citation Alignment	Deterministic Token Overlap	LLM-as-a-Judge (Real-time)	LLM checkers are slow, expensive, and can hallucinate. Algorithmic overlap checks are fast, local, and 100% deterministic.

Clinical Significance

By enforcing strict algorithmic bounds on LLM generations, X-CDS guarantees that patient treatment suggestions are traceable directly to medical literature. This framework establishes the standard for deploying generative models safely in clinical settings.